

2.1. Planificación

- **Objetivo.** Aprender a usar las instrucciones RISC-V para implementar bucles y condicionales. Conocer cómo la pila permite el almacenamiento de datos de forma temporal, así como la implementación de funciones.
- **Temporización:**
 - **Trabajo en laboratorio:** descripción de las instrucciones implicadas en la comparación y saltos. Uso de las mismas para realizar ejecución condicional y repetitiva (codificación de bucles). Uso de la pila e instrucciones para la implementación de subrutinas. Resolución de dudas.
 - **Trabajo autónomo:** ejercicios de recorrer vectores de valores realizando algún tipo de operación con ellos. Implementación de diversas funciones.
 - **Validación en laboratorio:** test y ejercicios evaluables.

2.1	Planificación	23
2.2	Introducción	23
2.3	Instrucciones de salto	23
2.4	Otras instrucciones RV32IM	26
2.5	Aspectos avanzados	27
2.6	Ejercicios propuestos	30

2.2. Introducción

Un programa que se ejecuta de manera secuencial, de principio a fin, ha de ser necesariamente simple y siempre realizará la misma tarea¹. Para llevar a cabo funciones más complejas, será preciso contar con herramientas de control del flujo de ejecución, a saber:

- La posibilidad de ejecutar o dejar de ejecutar un cierto bloque de instrucciones según que se cumpla o no una determinada condición, de forma similar a las instrucciones `if (cond) cierto else falso` de los lenguajes de programación de alto nivel.
- Tener la capacidad de repetir la ejecución de un bloque de instrucciones las veces que se precise, como lo hacen instrucciones del tipo `for` o `while` en lenguajes de alto nivel.

En ambos casos se depende de un mecanismo fundamental en la programación de ordenadores: la alteración del **flujo de ejecución**. Esto implica cambiar de alguna forma el contenido del **contador de programa**², para que no se limite a ejecutar una instrucción tras otra en cada ciclo de su funcionamiento que, como sabemos, es el comportamiento por defecto.

1: Cada vez que se ejecute procesará las mismas instrucciones aplicadas sobre los mismos datos o posiciones de memoria.

2: En RISC-V el contador de programa es el registro **pc**, cuyo valor se actualiza de forma automática a medida que se ejecutan las instrucciones de un programa.

2.3. Instrucciones de salto

Aquí entran en escena las conocidas como instrucciones de salto. Estas modifican el contenido del **pc** asignándole un valor que, por regla general, estará simbolizado por una etiqueta dispuesta en el código fuente del programa. Esto nos evitará tener que calcular manualmente las direcciones de las distintas partes de nuestro programa.

Cambiar el contenido del **pc** conlleva ejecutar las instrucciones dispuestas en memoria a partir de la dirección asignada, en lugar de la instrucción siguiente como correspondería. Antes de realizar ese cambio es posible guardar el actual contenido del **pc** en otro registro, por regla general con el objetivo de volver más tarde a ese mismo punto. De esta forma se facilita la ejecución de rutinas/funciones y el retorno desde las mismas.

De las instrucciones enumeradas en la [Tabla 1.2](#) —la tabla se encuentra en la página 10, con el título [Conjunto de instrucciones de la ISA RV32IM](#)— hay un grupo, en total ocho de ellas a las que habría que sumar varias seudoinstrucciones más con fines similares, que son las instrucciones de salto. Todas ellas se dividen en dos subgrupos: las de salto condicional y las de salto incondicional. Según ese criterio se detallan en los siguientes apartados.

El valor inmediato que todas estas instrucciones precisan como tercer argumento es un desplazamiento, positivo o negativo, que se sumará al contador de programa, de forma que si la condición se cumple se ejecuta la operación $pc \leftarrow pc + imm$. Todas estas instrucciones son del tipo S (véase la Figura 1.12 del primer capítulo) lo cual implica que el desplazamiento está limitado a 12 bits.

IMPLEMENTACIÓN DE UNA ESTRUCTURA CONDICIONAL

2.3.1. Saltos condicionales

Esta categoría de instrucciones se caracteriza por llevar a cabo una operación aritmético-lógica, de comparación entre dos valores, según cuyo resultado se alterará o no el contador de programa. Las instrucciones disponibles son:

- **beq** rs1, rs2, imm: salta si $rs1 = rs2$.
- **bne** rs1, rs2, imm: salta si $rs1 \neq rs2$.
- **bge** rs1, rs2, imm: salta si $rs1 \geq rs2$.
- **bgeu** rs1, rs2, imm: como la anterior pero interpretando los valores sin signo.
- **blt** rs1, rs2, imm: salta si $rs1 < rs2$.
- **bltu** rs1, rs2, imm: como la anterior pero interpretando los valores sin signo.

La igualdad o desigualdad de los dos valores, alojados en sendos registros, es indistinta de si estos se interpretan como números con o sin signo. Al comparar si uno es menor o mayor que el otro, sin embargo, sí hay que tomar en consideración la existencia de signo, de ahí que haya disponibles dos versiones.

Gracias a estas instrucciones resulta posible implementar condicionales tal y como se aprecia en el Listado 2.1, en el que se ejecuta una instrucción u otra según que el valor en **a2** sea mayor que el de **a1** o no. En la versión en ensamblador faltaría situar la etiqueta **fin**, desde la que continuaría la ejecución del resto del código del programa.

Listado 2.1: Un condicional escrito en ensamblador RISC-V (izqda.) y en lenguaje C.

<pre> 1 bge a2, a1, mayora2 2 sub a0, a1, a2 3 j fin 4 mayora2: 5 sub a0, a2, a1 </pre>	<pre> 1 if (a1 > a2) 2 a0 = a1 - a2; 3 else 4 a0 = a2 - a1; 5 </pre>
---	---

NOTA. En estos y otros fragmentos de código no se incluyen las directivas que marcan el inicio de la sección de código o datos, ni tampoco las dos instrucciones encargadas de poner fin al programa. Para ejecutar estos ejemplos sería preciso completar esas partes a fin de tener un programa completo.

IMPLEMENTACIÓN DE UN BUCLE

Sobre la base de las anteriores instrucciones, RISC-V define múltiples pseudoinstrucciones que facilitan saltos con otras condiciones, como **bgt**, **bgtu**, **ble** y **bleu**, que toman los mismos argumentos que las instrucciones previas, y **beqz**, **bnez**, **bgez**, **blez** y **bgtz**, en las que el segundo argumento es siempre el registro **zero**. Introduce estas pseudoinstrucciones en el editor de Ripes y observa en el panel derecho el código real que generan.

De manera análoga podríamos crear una estructura de repetición o bucle, para lo cual nos serviríamos de un salto condicional que volvería al inicio del cuerpo del bucle hasta que deje de cumplirse una cierta relación que, en el caso del Listado 2.2, es que el contenido de **a0** no sea cero.

Listado 2.2: Un bucle escrito en ensamblador RISC-V (izqda.) y en lenguaje C.

<pre> 1 li a0, 10 2 li a7, 1 3 repite: 4 ecall 5 addi a0, a0, -1 6 bnez a0, repite </pre>	<pre> 1 for(int a0=10; a0 >= 0; a0--) 2 std::cout << a0; 3 </pre>
---	--

La pseudoinstrucción **bnez** compara el contenido de **a0** con el registro **zero** y, si no coinciden, ejecuta el salto, de forma que el bucle se repetirá hasta que **a0** contenga el valor 0.

2.3.2. Saltos incondicionales

Como su propia denominación indica, estas instrucciones cambian el valor del contador de programa siempre que se ejecutan, sin que exista condición previa alguna. Hay dos instrucciones de salto incondicional disponibles:

- **jal** rd, imm: es una instrucción de tipo U³ que cuenta con dos operandos. En rd se guardará el actual contenido del **pc**, tras lo cual este se actualizará sumándole el desplazamiento de 20 bits entregado como segundo argumento.
- **jalr** rd, rs, imm: es una instrucción de tipo I⁴ que cuenta con tres operandos. En rd se guardará el actual contenido del **pc**, tras lo cual se actualizará asignándole el resultado de la suma **rs + imm**. En la práctica esto permite saltar a cualquier punto de un programa, sin las limitaciones de los desplazamientos de 12 o 20 bits respecto a la posición actual del **pc**.

3: La estructura de las instrucciones tipo U es la mostrada en la Figura 1.11 (pág. 18) y cuenta con un registro de destino y un valor inmediato de 20 bits.

4: La estructura de las instrucción tipo I es la mostrada en la Figura 1.10 (pág. 18) y cuenta con un registro de destino, uno de origen y un valor inmediato de 12 bits.

Como primer parámetro de estas instrucciones suele emplearse el registro **ra** (seudónimo de **x1**). La seudoinstrucción **jal imm** produce el código **jal x1, imm**, de forma que no es preciso indicar de manera explícita el registro de destino. También es posible saltar de manera incondicional a un punto del programa con la seudoinstrucción **j imm**, caso en el que se genera el código **jal x0, imm** y, por tanto, el valor actual en el **pc** es descartado.

La diferencia fundamental entre **jal** y **jalr** estriba en que esta última no suma el desplazamiento al **pc**, sino que asigna a este el resultado de **rs + imm**. Dado que antes se puede asignar a **rs** cualquier valor⁵, esta instrucción hace posible la transferencia de la ejecución a cualquier punto del espacio de direccionamiento, sin las limitaciones de 12 (rango de 4 KBytes) o 20 bits (rango de 1 MByte) que imponen los desplazamientos relativos.

5: Lo habitual es emplear instrucciones como **lui** y **auipc**, que ya conocemos, con este fin.

2.3.3. Llamadas a subrutinas

El hecho de que ambas instrucciones de salto incondicional, **jal** y **jalr**, almacenen el actual valor del **pc** en un registro antes de cambiarlo, hace posible el retorno a ese mismo punto⁶. No hay más que usar la instrucción **jalr** y cambiar el registro de destino por el de origen, usando el valor 0 como desplazamiento para que la operación real sea **pc ← rs**.

Las seudoinstrucciones **call** y **ret** facilitan este tipo de operativa. La primera prepara en el registro **ra** la dirección base, que será la del **pc**, y salta al punto indicado. La segunda toma de **ra** la dirección de retorno, enviando a **x0** el actual valor del **pc**.

6: En realidad se toma el valor del **pc**, se le suma 4 y, a continuación, se guarda en el registro de destino, de forma que esa dirección apunta a la instrucción siguiente a la actual.

ESCRITURA Y LLAMADA A UNA SUBROUTINA

El Listado 2.3 es un programa en el que se ha escrito una subrutina, a partir de la etiqueta **print**, a la que se llama múltiples veces desde el interior de un bucle. Se usan las llamadas del sistema ofrecidas por Ripes para mostrar cadenas de caracteres y números tanto en decimal como en hexadecimal.

```

1  .data
2  dec: .string "Decimal: "
3  hex: .string "Hexadecimal: "
4  sep: .string " - "
5  endl: .string "\n"
6
7  .text
8  inicio:
9      li a1, 9
10     li a2, 20
11
12     bucle:
13         call print
14         addi a1, a1, 1
15         ble a1, a2, bucle
16         li a7, 10 # Terminar
17         ecall
18
19     print:
20         la a0, dec
21         li a7, 4
22         ecall
23         mv a0, a1
24         li a7, 1
25
26     ecall
27     la a0, sep
28     li a7, 4
29     ecall
30     la a0, hex
31     ecall
32     mv a0, a1
33     li a7, 34
34     ecall
35     la a0, endl
36     li a7, 4
37     ecall
38     ret

```

Listado 2.3: Programa con una rutina de impresión de números en decimal y hexadecimal.

Prueba el programa en Ripes para observar la salida que produce. Cambia los límites del bucle y experimenta con otras alternativas de impresión.

Dado que la seudoinstrucción **call** siempre almacena la dirección de retorno en el registro **ra**, de donde la toma la seudoinstrucción **ret** para volver a asignarla al **pc**, no es posible usar estas seudoinstrucciones para tener subrutinas anidadas, es decir, una función que llame a otra, porque el primer valor de **ra** se perdería y, con él, la posibilidad de devolverle el control al punto adecuado.

2.4. Otras instrucciones de la arquitectura RV32IM

El conjunto de instrucciones que conocemos hasta el momento —aritméticas básicas, carga y almacenamiento en memoria, saltos condicionales y saltos incondicionales— son suficientes para escribir la gran mayoría de programas. Sin embargo, esto no significa que contar con instrucciones adicionales no pueda facilitarnos la tarea en ciertas ocasiones. En este apartado se enumeran buena parte de ellas.

2.4.1. Instrucciones lógicas

Las operaciones lógicas entre bits son de uso muy habitual al programar a bajo nivel, ya que con ellas es posible activar un bit concreto, desactivarlo, invertirlo, comprobar si está activo o no, etc. Las instrucciones disponibles para ello son las siguientes:

- **and** rd, rs1, rs2 y **andi** rd, rs1, imm: efectúa un *AND* bit a bit entre los operandos rs1 y rs2 —la primera— o bien rs1 e imm —la segunda—, dejando el resultado en rd.
- **or** rd, rs1, rs2 y **ori** rd, rs1, imm: efectúa un *OR* bit a bit entre los dos operandos y guarda el resultado en rd.
- **xor** rd, rs1, rs2 y **xori** rd, rs1, imm: efectúa un *XOR* bit a bit entre los dos operandos y guarda el resultado en rd.

A estas hay que añadir un grupo de instrucciones que también operan al nivel de bit, pero en este caso con el objetivo de producir desplazamientos en un sentido u otro:

- **sll** rd, rs1, rs2 y **slli** rd, rs1, imm: desplaza el valor contenido en rs1 hacia la izquierda, tantos bits como indique rs2 —la primera— o imm —la segunda—, dejando el resultado en rd.
- **srl** rd, rs1, rs2 y **srl**i rd, rs1, imm: desplaza el valor contenido en rs1 hacia la derecha, tantos bits como indique rs2 o imm, y guarda el resultado en rd.
- **sra** rd, rs1, rs2 y **srai** rd, rs1, imm: desplaza el valor contenido en rs1 hacia la derecha, tantos bits como indique rs2 o imm, tomando en consideración el bit de signo. El resultado se almacena en rd.

2.4.2. Instrucciones de comparación

En el apartado [Saltos condicionales](#) (pág. 24) hemos conocido instrucciones que verifican si se cumple una cierta relación entre dos registros y, en consecuencia, alteran el flujo de ejecución⁷. Además de esas, en RV32IM disponemos de cuatro instrucciones de comparación adicionales. La diferencia es que estas no alteran el valor del **pc**, sino que se limitan a almacenar el valor 1 o 0 en un registro de destino, según que el primer operando sea menor que el segundo o no, respectivamente.

Realmente es una sola instrucción con cuatro variantes diferentes: **slt** rd, rs1, rs2, **sltu** rd, rs1, rs2, **slti** rd, rs1, imm y **sltiu** rd, rs1, imm. En todos los casos se comprueba si el contenido del registro rs1 es menor que el almacenado en rs2 o el valor inmediato imm. Las dos primeras operan con números con signo, mientras que las dos últimas lo hacen con números sin signo. Si intercambiamos el orden de los operandos a comparar, sería equivalente a tener una instrucción **sge**.

Estas instrucciones son útiles cuando el resultado de la comparación no necesariamente implica cambiar el flujo de ejecución. Imaginemos que tenemos una lista de calificaciones y, recorriéndolas con un bucle, queremos contar cuántas de ellas son inferiores a 5.

¿Cómo comprobarías si el valor almacenado en un registro es par o impar? Bastaría con realizar una operación lógica *AND* con el valor adecuado. Todos los números impares tienen el bit de menor peso a 1, mientras que en los pares está a 0. Con **andi** t0, a0, 1 tendríamos en t0 el valor 0 si el número es par o 1 en caso contrario, por lo que con un salto condicional **beqz** podríamos discernir entre ambas posibilidades.

Las instrucciones **sll** y **srl** llevan a cabo un desplazamiento lógico de los bits, tratándolos a todos por igual, mientras que **sra** trata de forma especial el bit de signo, por lo que se dice que efectúa un **desplazamiento aritmético** en lugar de lógico.

El desplazamiento de un valor en un bit a la izquierda equivale a multiplicar dicho valor por dos. De forma análoga, desplazarlo a la derecha resulta en una división por dos. Cambiando el número de bits es posible multiplicar/dividir por cualquier potencia de dos sin necesidad de recurrir a las instrucciones **mul** y **div**. Esto resulta interesante porque el desplazamiento de bits es mucho más eficiente.

7: Muchas arquitecturas de procesador no disponen de instrucciones así, sino que es preciso primero comparar dos operandos y obtener un señalizador, un valor que suele ser 0 o 1, en base al cual se decide con una segunda instrucción si se salta o no.

CONTAR NÚMERO DE VALORES MENORES A UNO DADO

Podríamos hacerlo con instrucciones de salto, para comparar si el valor cargado desde memoria es inferior a 5 y, en consecuencia, desviar la ejecución a una instrucción que incremente un contador. La alternativa sería la mostrada en el Listado 2.4, en la que no existe dicho salto condicional.

```

1 | .data
2 |
3 | notas: .word 7, 5, 6, 3, 8, 2, 4, 6, 4, 9
4 |
5 | .text
6 |
7 | la t0, notas
8 | li t1, 10
9 | li a0, 0
10 |
11 | bucle:
12 | lw a2, 0, t0
13 | slti a1, a2, 5
14 | add a0, a0, a1
15 |
16 | addi t1, t1, -1
17 | addi t0, t0, 4
18 | bnez t1, bucle

```

En una práctica posterior, dedicada a los riesgos de control en la ejecución en un cauce segmentado, apreciaremos la importancia de evitar las instrucciones de salto siempre que sea posible.

Listado 2.4: Contar el número de valores que son inferiores a uno dado.

2.4.3. Otras pseudoinstrucciones

En esta y la práctica anterior hemos conocido la mayoría de pseudoinstrucciones contempladas en la ISA RV32IM. En realidad, solo hay tres más de uso relativamente común:

- **not** rd, rs: calcula el complemento a uno⁸ de rs y lo almacena en rd.
- **neg** rd, rs: calcula el complemento a dos⁹ de rs y lo almacena en rd.
- **nop**: no realiza ningún trabajo útil, simplemente consume uno o más ciclos de reloj.

8: Recuerda que el complemento a uno de un valor se obtiene invirtiendo el estado de todos sus bits.

9: El complemento a dos se genera como el complemento a uno más 1.

2.5. Aspectos avanzados

Ya conocemos las instrucciones de salto, mediante las cuales es posible realizar llamadas a funciones desde cualquier punto de un programa y, cuando terminan su ejecución, volver a la instrucción correspondiente¹⁰. Este es un aspecto fundamental para los lenguajes de alto nivel y, en consecuencia, nos interesa saber cómo ajustar nuestro código ensamblador en caso de que tuviese que interactuar con código producido por un compilador de C, por poner un ejemplo¹¹.

10: Con instrucción de salto, pero en este caso usando como dirección de destino la que se guardó en el registro **ra** al efectuar el primer salto.

11: En el capítulo 11. Interfaz entre ensamblador y C profundizaremos en este aspecto.

Además de la instrucción de salto incondicional, encargada de llevar el punto de ejecución al inicio de la función a la que se invoca, los lenguajes de alto nivel siguen unas *convenciones de llamada* que determinan, entre otros aspectos, los registros que han de utilizarse para facilitar parámetros a una función, al llamarla, o los empleados para devolver los valores de retorno. Es un escenario en el que se utiliza una estructura de datos básica: la pila, cuya situación en el mapa de memoria se representó en la Figura 1.7 (véase la página 16).

2.5.1. Almacenamiento temporal de datos en la pila

Cuando se llama a una función es con el objetivo de que realice un cierto trabajo. Para ello, se verá obligada a usar algunos registros. No obstante, el código de la función debe tener presente que el programa desde el que se la ha llamado espera que el contenido de determinados registros no cambie, por lo que no debería alterarlos o, en su defecto, debe preservarlos a fin de restaurarlos antes de devolver el control.

Una función puede modificar cualquiera de los registros temporales: **t0** a **t6**. Asimismo, es libre también de alterar los registros **a0** a **a7** que, como se ha explicado, sirven como parámetros de llamada y retorno. Si el código que invoca a una función tiene interés en el contenido de alguno de esos registros, debe encargarse por sí mismo de preservarlos. Todos los demás han de ser conservados, lo que implica que la función debe, en caso de que necesite utilizarlos, guardar su contenido y restaurarlo al final de la misma.

Colocando el puntero del ratón sobre cualquier registro, en el panel GPR de Ripes, aparecerá una descripción de su finalidad y, en una segunda línea, la indicación **SAVER: CALLER** o **SAVER: CALLEE**. En el primer caso el registro ha de ser preservado por el código que llama a la función, mientras que en el segundo es esta la asume esa responsabilidad.

El registro **sp** contiene siempre la dirección de la parte alta de la pila —la de la primera posición libre— en una estructura que crece en memoria hacia abajo, a medida que en ella se guardan datos.

12: Las ISA de tipo CISC, como es el caso de x86/x64, disponen de instrucciones específicas para guardar y recuperar datos de la pila, con nombres como **push** y **pop**. Estas se encargan tanto de ajustar el valor del puntero de pila, para lo cual existe un registro dedicado, como de escribir o leer de memoria el contenido del registro a preservar.

13: En realidad, la función no necesita cambiar todos esos registros, pero supondremos que es así para aprender a utilizar la pila con este fin.

LA FUNCIÓN **pow(base,exp)**

Listado 2.5: Función **pow()**, a partir de la etiqueta **pow**, y programa que la usa, a partir de la etiqueta **main**.

```

1  main:
2      li a0, 2
3      li a1, 8
4
5      call pow # Calcular 2^8
6
7      li a7, 1 # Mostrar por consola y terminar
8      ecall
9
10     li a7, 10
11     ecall
12
13     # Función powr(base, exponente)
14     # -> a0 = resultado
15 pow:
16     addi sp, sp, -32 # Reservar espacio en pila
17     sw ra, 0, sp # Preservar registros
18     sw gp, 4, sp
19     sw s0, 8, sp
20
21     sw s1, 12, sp
22     sw s2, 16, sp
23
24     li s0, 1 # Trabajo de la función
25     bucpow:
26     beqz a1, finpow
27     mul s0, s0, a0
28     addi a1, a1, -1
29     j bucpow
30
31     finpow:
32     mv a0, s0
33
34     lw s2, 16, sp # Recuperación de registros
35     lw s1, 12, sp
36     lw s0, 8, sp
37     lw gp, 4, sp
38     lw ra, 0, sp
39     addi sp, sp, 32 # Dejar sp como estaba
40     ret

```

2.5.2. La pila para transferir parámetros a funciones

El número de registros con que cuenta cualquier CPU es limitado, de ahí que sea preciso recurrir a la memoria para almacenar los datos. Esto es algo habitual cuando, por ejemplo, se invoca a una función con un gran conjunto de parámetros. Es una situación donde, de nuevo, se emplea la pila.

Hasta ocho registros, de **a0** a **a7**, están destinados a facilitar la transferencia de parámetros al invocar a una función¹⁴. Se usan por orden, de manera que si una función solo precisa dos argumentos, estos se entregarán en los registros **a0** y **a1**. En el caso de una función tomase como entrada más de ocho valores, los ocho primeros se asignarían a los registros citados, mientras que el noveno y siguientes se alojarían en la pila.

El programa del **Listado 2.6** define una función cuya finalidad es calcular el valor promedio —entero, sin parte decimal— de una lista de diez valores entregados como argumentos. Hay dos aspectos a destacar en su código:

14: Si la CPU cuenta con la extensión **F** o **D**, para operar con números en punto flotante, también se dispondría de los registros **fa0** a **fa7** con la misma finalidad, pero para números de tipos en punto flotante.

- El código del cuerpo principal, que ocupa la columna izquierda del listado, se encarga de reservar en la pila la memoria necesaria para enviar los argumentos adicionales, en este caso dos valores enteros. Para ello ajusta la dirección del registro **sp**, introduce los valores en el espacio reservado, invoca a la función y, por último, reajusta **sp** para liberar el espacio¹⁵.
- La función `avg(n1, n2, ...)` necesita usar también la pila, a fin de guardar los registros que alterará para realizar su trabajo. Por esta razón modifica el registro **sp** al inicio. No obstante, precisa del valor que tenía dicho registro cuando fue invocada, para así poder acceder a los parámetros de llamada que no vienen en los registros. Esta es la razón de que se asigne a **s0** el valor que tenía originalmente **sp** —con la instrucción `addi s0, sp, 16`— y se use esa dirección para leer los dos argumentos adicionales.

15: Dado que esos dos argumentos eran solo de salida, enviados a la función pero sin que sirvan para devolver resultado alguno, en el cuerpo principal no se leen antes de liberar el espacio de la pila.

LA FUNCIÓN `avg(n1,n2,...)`

Listado 2.6: Función `avg()`, a partir de la etiqueta `avg`, y programa que la usa, a partir de la etiqueta `main`.

```

1  main:
2      li a0, 7
3      li a1, 12
4      li a2, 5
5      li a3, 1
6      li a4, 17
7      li a5, 3
8      li a6, 10
9      li a7, 11
10
11     # Reserva de espacio en la pila
12     # para los dos parámetros adicionales
13     addi sp, sp, -16
14     li t0, 4
15     sw t0, 0, sp
16     li t0, 15
17     sw t0, 4, sp
18
19     call avg
20
21     # Se descarta el espacio anterior
22     addi sp, sp, 16
23
24     li a7, 1
25     ecall
26
27     li a7, 10
28     ecall
29
30     # Función avg
31 avg:
32     addi sp, sp, -16
33     sw s0, 0, sp
34     sw s1, 4, sp
35
36     # Se usará s0 como puntero
37     # de marco (fp) para acceder
38     # a parámetros adicionales
39     addi s0, sp, 16
40
41     add s1, a0, a1
42     add s1, s1, a2
43     add s1, s1, a3
44     add s1, s1, a4
45     add s1, s1, a5
46     add s1, s1, a6
47     add s1, s1, a7
48
49     lw t0, 0, s0
50     add s1, s1, t0
51     lw t0, 4, s0
52     add s1, s1, t0
53     li s0, 10
54     div a0, s1, s0
55
56     lw s1, 4, sp
57     lw s0, 0, sp
58     addi sp, sp, 16
59
60     ret

```

NOTA. Al examinar el código de la función `avg(n1, n2, ...)` podríamos preguntarnos por qué no se asigna directamente el valor del registro **sp** a **s0** antes de cambiar el primero. La razón es que, en ese caso, perderíamos el contenido de **s0**, un registro que debe ser preservado por la función a la que se invoca. Se podría haber empleado un registro temporal, como **t0**, para mantener la dirección original de **sp**, pero por convención es habitual usar **s0** —también conocido como registro **fp** (*frame pointer*)— para tal tarea.

2.5.3. Funciones recursivas y la pila

Cuando se escribe una función en un lenguaje de alto nivel, como podría ser la del Listado 2.7, el compilador se ocupa de generar en cada llamada a la misma lo que se conoce como un *stack frame*. En cada llamada a `factorial()` hay que guardar en la pila la lista de parámetros recibidos, en este caso solo el valor entero `n`, y la dirección de retorno de la llamada. En una implementación recursiva, como esta que se propone, esa operación se efectuará múltiples veces hasta llegar al caso de parada, momento en el que van extrayéndose de la pila y devolviendo a los registros sus valores originales.

```

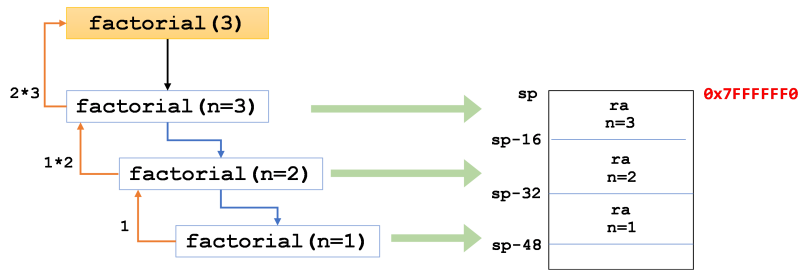
1  int factorial(int n) {
2      if (n == 1)
3          return n;
4      else
5          return n * factorial(n - 1);
6  }

```

Listado 2.7: Función factorial escrita en lenguaje C que usa recursividad.

IMPLEMENTAR UNA FUNCIÓN RECURSIVA

Figura 2.1: Evolución del contenido de la pila durante las llamadas recursivas a la función. En la primera llamada se guarda en la parte alta de la pila la dirección de retorno —registro **ra**— y el valor actual del parámetro de entrada. Esta operación se repite en cada una de las llamadas, de forma que en la pila van conservándose los datos correspondiente al ámbito de cada ejecución de la función. Al llegar al caso base se recuperarán de la pila los datos en sentido inverso, de forma que se devuelve el valor que en ese momento tenía el parámetro y se restablece en **ra** la dirección de retorno.



Un compilador de C produce el código ensamblador preciso para ir ajustando el registro **sp**, almacenar y extraer datos de la pila. Ese es un código que tendremos que escribir de forma manual para conseguir la mencionada funcionalidad. El **Listado 2.8** muestra cómo implementar la función anterior en RISC-V.

Listado 2.8: Código RISC-V que implementa la función factorial de forma recursiva.

```

1 # Entrada: a0 -> Número del que se quiere calcular el factorial
2 # Salida: a1 -> Resultado
3 factorial:
4     addi sp, sp, -16 # Reservar 8 bytes de espacio en la pila
5     sw ra, 0(sp)     # Para guardar la dirección de retorno
6     sw a0, 4(sp)     # y el actual valor de a0
7
8     addi a0, a0, -1
9     bnez a0, recur   # return 1 if a0 = 0
10    li a1, 1          # else a0 * factorial(a0-1)
11    j finrecur
12
13 recur:
14     call factorial   # Llamada recursiva
15
16 finrecur:
17     lw a0, 4(sp)     # Recuperar el anterior valor de a0
18     mul a1, a1, a0    # y multiplicar a1 por a0
19     lw ra, 0(sp)     # Recuperar dirección de retorno
20     addi sp, sp, 16   # Devolver espacio en la pila
21     ret              # Retornar

```

Introduce el código de esta función en Ripes. Delante del mismo tendrás que inicializar el registro **a0** con el valor que quieras, llamar a la función con **call** y, por último, mostrar por consola el contenido de **a1**. Hay que evitar que el flujo de ejecución entre en la función sin haberla invocado con **call**, caso en el que la pila no contendría un valor de retorno válido. Para ello usa la instrucción **ecall** con la función adecuada para finalizar el programa.

Las tres primeras líneas de código son las encargadas de crear el *stack frame* y guardar en la pila la dirección de retorno y valor actual del parámetro de entrada. Después se actualiza ese parámetro y, con un condicional, se determina si se ha alcanzado el caso base. De no ser así, se produce una nueva llamada a la misma función. Al llegar al caso base se recupera de la pila el parámetro, se realiza la operación que corresponde —en este caso una multiplicación— y, por último, se extrae la dirección de retorno y se reajusta la pila.

2.6. Ejercicios propuestos

Los ejercicios propuestos a continuación te permitirán familiarizarte con las instrucciones condicionales y de salto, poniéndolas en práctica en diversos casos prácticos y preparándote para la evaluación de esta segunda práctica.

CODIFICAR BUCLES

Ejercicio 2.1. Sumar los valores 1 a 1000, acumulando la suma en el registro **a0**, y mostrar el resultado por la consola.

Ejercicio 2.2. Sumar los valores impares entre 1 y 1000 en el registro **a0** y mostrar el resultado por la consola.

Ejercicio 2.3. Mostrar por la consola los números 40 a 122 y el carácter que corresponde a cada uno de esos códigos, para conseguir un resultado similar al de la imagen inferior. Consulta las llamadas del sistema de Ripes para saber cómo imprimir el carácter.

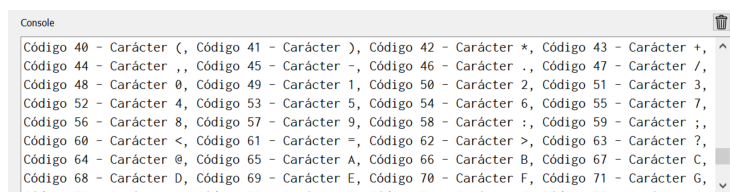
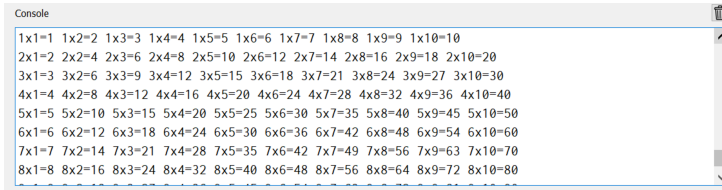


Figura 2.2: Vista parcial de la tabla de códigos y caracteres generada por el programa.

Ejercicio 2.4. Imprime por la consola las tablas de multiplicar de los números a 1 a 10 con un aspecto similar al de la imagen inferior.



```

Console
1x1=1 1x2=2 1x3=3 1x4=4 1x5=5 1x6=6 1x7=7 1x8=8 1x9=9 1x10=10
2x1=2 2x2=4 2x3=6 2x4=8 2x5=10 2x6=12 2x7=14 2x8=16 2x9=18 2x10=20
3x1=3 3x2=6 3x3=9 3x4=12 3x5=15 3x6=18 3x7=21 3x8=24 3x9=27 3x10=30
4x1=4 4x2=8 4x3=12 4x4=16 4x5=20 4x6=24 4x7=28 4x8=32 4x9=36 4x10=40
5x1=5 5x2=10 5x3=15 5x4=20 5x5=25 5x6=30 5x7=35 5x8=40 5x9=45 5x10=50
6x1=6 6x2=12 6x3=18 6x4=24 6x5=30 6x6=36 6x7=42 6x8=48 6x9=54 6x10=60
7x1=7 7x2=14 7x3=21 7x4=28 7x5=35 7x6=42 7x7=49 7x8=56 7x9=63 7x10=70
8x1=8 8x2=16 8x3=24 8x4=32 8x5=40 8x6=48 8x7=56 8x8=64 8x9=72 8x10=80
  
```

Figura 2.3: Vista parcial de las tablas de multiplicar mostradas por el programa.

Ejercicio 2.5. Almacenar en memoria un vector con los valores 7, 4, 23, 12, 6, 20, 17, 8, 3, 10 y mostrar por consola la media aritmética. Para ello se ha de calcular su suma y también contar cuántos valores hay en el vector.

BUCLES Y CONDICIONALES

Ejercicio 2.6. Almacenar en memoria un vector con los valores 7, 4, 23, 12, 6, 20, 17, 8, 3, 10 y escribir un programa que muestre por consola la suma de aquellos que ocupan posiciones impares, asumiendo que al primer 7 le corresponde la posición 1 (impar).

Ejercicio 2.7. Modifica el programa del ejercicio previo para que sume todos aquellos valores del vector que sean números impares, independientemente de la posición que ocupen en el vector, y muestre el resultado por la consola.

Ejercicio 2.8. Usando el mismo vector del ejercicio 2.6, escribe un programa para buscar el menor y el mayor valor almacenándolos en **a0** y **a1**, respectivamente. Muestra ambos valores por la consola.

Ejercicio 2.9. Almacenar en memoria el vector de valores 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16. Escribir un programa que lo trate como una matriz de 4x4, como en el listado de la derecha, y calcule la suma de cada una de las filas y las muestre por consola

```

1 | .word 1, 2, 3, 4
2 | .word 5, 6, 7, 8
3 | .word 9, 10, 11, 12
4 | .word 13, 14, 15, 16
  
```

Ejercicio 2.10. Modificar el programa anterior para que calcule y muestre la suma de cada una de las columnas de la matriz.

Ejercicio 2.11. Almacenar en memoria el vector de valores 6, 1, 6, 7, 2, 3, 6, -3, 6, 1, -2, 3, -5, 2, 6. Escribir un programa que cargue el primer valor del vector, cuente cuántas veces aparece en él y muestre el conteo por consola.

Ejercicio 2.12. Usa el mismo vector del ejercicio previo pero cambia el programa para que cuente cuántos valores negativos hay y lo muestre por consola.

IMPLEMENTAR FUNCIONES

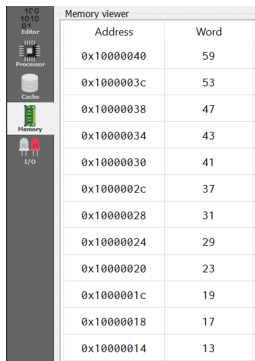
Ejercicio 2.13. Usa el vector de valores del ejercicio 2.9 interpretado como una matriz de 4x4. Escribe una función que tome como entrada una fila en **a1** y una columna en **a2** y devuelva en **a0** el valor contenido en esa posición de la matriz. La función solo podrá alterar el contenido de los registros temporales **t0-t6**. Llama a la función para mostrar por consola los valores en las posiciones (0,0), (1,1), (2,2) y (3,3).

Ejercicio 2.14. Completa el programa anterior con una función que tome como parámetros la fila en **a1**, la columna en **a2**, el valor en **a0** y muestre por consola una mensaje del tipo El valor en la fila a1 columna a2 es a0. Úsala para mostrar los cuatro elementos antes indicados. Como en el caso anterior, la función solo deberá alterar registros temporales. Desde el programa invoca primero a la función que obtiene el valor en **a0** y, posteriormente, a esta nueva función para mostrarlo en la consola.

Ejercicio 2.15. Escribe una función que tome como entrada —en el registro **a0**— un valor numérico y que muestre por consola la secuencia desde 0 hasta **a0**. Para ello la función operará de forma recursiva, de manera que ella misma se llamará con **a0 -1** una vez tras otra hasta alcanzar el caso base: que **a0** sea igual a 0. Utiliza

la pila para guardar tanto la dirección de retorno como los valores que tomará **a0** en cada llamada. Por último, llama a dicha función desde la parte inicial del programa probando con distintos valores.

```
1 int mcd(int a1, int a2) {
2     int a = a2 % a1, b = a1;
3
4     if (a == 0)
5         return b;
6     else
7         return mcd(a, b);
8 }
```



Address	Word
0x10000040	59
0x1000003c	53
0x10000038	47
0x10000034	43
0x10000030	41
0x1000002c	37
0x10000028	31
0x10000024	29
0x10000020	23
0x1000001c	19
0x10000018	17
0x10000014	13

Figura 2.4: Estado de la memoria tras ejecutar el programa.

Ejercicio 2.16. El máximo común divisor (MCD) de dos números se calcula de forma muy sencilla mediante un proceso recursivo como el que muestra el programa en lenguaje C mostrado al margen. Escribe una implementación análoga de forma que los registros **a1** y **a2** contengan los dos números de entrada y **a0** el MCD encontrado que, en último término, sería 1 si ambos números no comparten ningún otro factor. Llama desde el cuerpo del programa a la función con distintos valores para verificar su funcionamiento.

Ejercicio 2.17. Escribe un programa en el que inicialmente se tiene en memoria el vector primos: .word 2, 3, 5, 7, 11, 13 con los seis primeros números primos. El programa deberá completar la tabla, agregando los primos adicionales que encuentre hasta llegar al 100. Con este fin escribe una función que tomando como entrada un número —en el registro **a1**— y el número de primos que se tienen actualmente en el vector —en el registro **a2**— verifique si dicho número es primo o no. Para ello, no hay más que comprobar si el número en **a1** es divisible por cualquiera de los valores existentes en el vector. La función devolverá el valor 1 en el registro **a0** si el número es primo o 0 en caso contrario. Llama a la función desde un bucle en el cuerpo principal del programa, guardando cada nuevo primo en la tabla e incrementando **a2**. Usa el panel **MEMORY** para comprobar el vector en memoria, como se aprecia en la Figura 2.4. La función que comprueba si un número es primo puede escribirse tanto de forma iterativa como recursiva. ¿Qué versión es más compleja? ¿Qué versión es más eficiente?

Ejercicio 2.18. Escribe una función que, sin llevar a cabo ninguna operación funcional, provoque una espera de aproximadamente un segundo. Desde el cuerpo principal del programa muestra por consola la secuencia de valores 1 a 10 con una espera, de aproximadamente un segundo, entre un número y el siguiente.